

Experiment No: 5

Implement the Digital Signature algorithm using RSA approach or DSA approach

Name: Rutuja Dundesh Mane

Roll No: A44

Div: A

PRN: 2324000718

RSAExamServer.java:

```
import javax.swing.*.*;
import java.awt.*.*;
import java.io.*.*;
import java.net.*.*;
import java.security.*.*;
import java.security.spec.X509EncodedKeySpec;
import javax.crypto.Cipher;
import java.util.*.*;
import java.util.Base64;

public class RSAExamServer extends JFrame {

    private JTextArea logArea;
    private JTextArea serverPublicKeyBox;

    private ServerSocket serverSocket;
    private Map<String,String> qpDatabase;

    private KeyPair serverKeyPair;

    public RSAExamServer (){

        setTitle("RSA Algorithm");
        setSize(700,500);
        setDefaultCloseOperation(EXIT_ON_CLOSE);

        JPanel topPanel = new JPanel(new BorderLayout());

        serverPublicKeyBox = new JTextArea(4,50);
        serverPublicKeyBox.setBorder(BorderFactory.createTitledBorder("Server Public Key"));

        topPanel.add(new JScrollPane(serverPublicKeyBox),BorderLayout.CENTER);

        add(topPanel,BorderLayout.NORTH);

        logArea = new JTextArea();
        logArea.setBorder(BorderFactory.createTitledBorder("Server Activity Log"));
        add(new JScrollPane(logArea),BorderLayout.CENTER);
```

```

JButton startBtn = new JButton("Start Server (Port 8080)");
add(startBtn,BorderLayout.SOUTH);

startBtn.addActionListener(e -> startServer());

initializeServer();

setVisible(true);
}

private void initializeServer(){

    try{

        KeyPairGenerator gen = KeyPairGenerator.getInstance("RSA");
        gen.initialize(2048);
        serverKeyPair = gen.generateKeyPair();

        String pubKey = Base64.getEncoder()
            .encodeToString(serverKeyPair.getPublic().getEncoded());

        serverPublicKeyBox.setText(pubKey);

        qpDatabase = new HashMap<>();

        qpDatabase.put("10-CS101-1",
            "Q1. What is Java?\nQ2. Explain OOP.\nQ3. What is JVM?");

        log("Server RSA Keys Generated");

    }catch(Exception e){ e.printStackTrace(); }
}

private void startServer(){

    new Thread(() -> {

        try{

            serverSocket = new ServerSocket(8080);

            log("Server Started on Port 8080");

            while(true){

                Socket client = serverSocket.accept();
                log("Client Connected");

                handleClient(client);
            }
        }
    })

```

```

        }catch(Exception e){ log("Server Error: "+e.getMessage()); }

    }).start();
}

private void handleClient(Socket client){

    new Thread() -> {

        try{

            BufferedReader in =
                new BufferedReader(new InputStreamReader(client.getInputStream()));

            DataOutputStream out =
                new DataOutputStream(client.getOutputStream());

            String cls = in.readLine();
            String code = in.readLine();
            String sem = in.readLine();
            String clientPubKeyString = in.readLine();

            log("Received Client Public Key");

            byte[] keyBytes = Base64.getDecoder().decode(clientPubKeyString);

            KeyFactory kf = KeyFactory.getInstance("RSA");

            PublicKey clientPublicKey =
                kf.generatePublic(new X509EncodedKeySpec(keyBytes));

            String key = cls+"-"+code+"-"+sem;

            String qp = qpDatabase.get(key);

            if(qp==null){
                out.writeUTF("ERROR");
                return;
            }

            log("Generating Digital Signature using Server Private Key");

            Cipher signCipher = Cipher.getInstance("RSA");
            signCipher.init(Cipher.ENCRYPT_MODE, serverKeyPair.getPrivate());

            byte[] signature = signCipher.doFinal(qp.getBytes());

            log("Encrypting Exam Paper using Client Public Key");

            Cipher encryptCipher = Cipher.getInstance("RSA");
            encryptCipher.init(Cipher.ENCRYPT_MODE, clientPublicKey);

```

```

        byte[] encryptedQP = encryptCipher.doFinal(qp.getBytes());

        out.writeUTF("OK");

        out.writeInt(encryptedQP.length);
        out.write(encryptedQP);

        out.writeInt(signature.length);
        out.write(signature);

        String serverPub =
            Base64.getEncoder().encodeToString(
                serverKeyPair.getPublic().getEncoded());

        out.writeUTF(serverPub);

        log("Encrypted Question Paper Sent to Client");

        client.close();

    }catch(Exception e){
        log("Client Error: "+e.getMessage());
    }

}).start();
}

private void log(String msg){

    logArea.append(msg+"\n");
}

public static void main(String[] args){

    SwingUtilities.invokeLater(RSAExamServer::new);
}
}

```

RSAExamClient.java:

```

import javax.swing.*.*;
import java.awt.*.*;
import java.io.*;
import java.net.*;
import java.security.*;
import java.security.spec.X509EncodedKeySpec;
import javax.crypto.Cipher;
import java.util.Base64;

public class RSAExamClient extends JFrame {

```

```

private JTextField classField,courseField,semField;

private JTextArea clientPublicKeyBox;
private JTextArea encryptedBox;
private JTextArea signatureBox;
private JTextArea decryptedBox;
private JTextArea serverPublicKeyBox;
private JLabel verifyLabel;

private KeyPair clientKeyPair;

private byte[] encryptedQP;
private byte[] signature;
private PublicKey serverPublicKey;

public RSAExamClient (){

    setTitle("RSA Algrithm Client");
    setSize(750,650);
    setDefaultCloseOperation(EXIT_ON_CLOSE);

    JPanel inputPanel = new JPanel(new GridLayout(4,2));

    inputPanel.setBorder(BorderFactory.createTitledBorder("Exam Request"));

    inputPanel.add(new JLabel("Class"));
    classField = new JTextField("10");
    inputPanel.add(classField);

    inputPanel.add(new JLabel("Course Code"));
    courseField = new JTextField("CS101");
    inputPanel.add(courseField);

    inputPanel.add(new JLabel("Semester"));
    semField = new JTextField("1");
    inputPanel.add(semField);

    JButton requestBtn = new JButton("Request Exam");
    JButton decryptBtn = new JButton("Decrypt & Verify");

    inputPanel.add(requestBtn);
    inputPanel.add(decryptBtn);

    add(inputPanel,BorderLayout.NORTH);

    JPanel centerPanel = new JPanel(new GridLayout(5,1));

    clientPublicKeyBox = new JTextArea();
    clientPublicKeyBox.setBorder(BorderFactory.createTitledBorder("Client Public Key"));

```

```

        encryptedBox = new JTextArea();
        encryptedBox.setBorder(BorderFactory.createTitledBorder("Encrypted Question Paper Received"));

        signatureBox = new JTextArea();
        signatureBox.setBorder(BorderFactory.createTitledBorder("Digital Signature Received"));

        serverPublicKeyBox = new JTextArea();
        serverPublicKeyBox.setBorder(BorderFactory.createTitledBorder("Server Public Key"));

        decryptedBox = new JTextArea();
        decryptedBox.setBorder(BorderFactory.createTitledBorder("Decrypted Question Paper"));

        centerPanel.add(new JScrollPane(clientPublicKeyBox));
        centerPanel.add(new JScrollPane(encryptedBox));
        centerPanel.add(new JScrollPane(signatureBox));
        centerPanel.add(new JScrollPane(serverPublicKeyBox));
        centerPanel.add(new JScrollPane(decryptedBox));

        add(centerPanel, BorderLayout.CENTER);

        verifyLabel = new JLabel("Verification Status: Waiting...");
        verifyLabel.setHorizontalAlignment(SwingConstants.CENTER);
        verifyLabel.setFont(new Font("Arial", Font.BOLD, 16));

        add(verifyLabel, BorderLayout.SOUTH);

        generateClientKeys();

        requestBtn.addActionListener(e -> requestExam());
        decryptBtn.addActionListener(e -> decryptAndVerify());

        setVisible(true);
    }

    private void generateClientKeys(){

        try{

            KeyPairGenerator gen = KeyPairGenerator.getInstance("RSA");
            gen.initialize(2048);

            clientKeyPair = gen.generateKeyPair();

            String pub =
                Base64.getEncoder().encodeToString(
                    clientKeyPair.getPublic().getEncoded());

            clientPublicKeyBox.setText(pub);

        }catch(Exception e){ e.printStackTrace(); }
    }

```

```
}
```

```
private void requestExam(){
```

```
try{
```

```
    Socket socket = new Socket("localhost",8080);
```

```
    PrintWriter out =  
        new PrintWriter(socket.getOutputStream(),true);
```

```
    DataInputStream in =  
        new DataInputStream(socket.getInputStream());
```

```
    out.println(classField.getText());  
    out.println(courseField.getText());  
    out.println(semField.getText());
```

```
    out.println(clientPublicKeyBox.getText());
```

```
    String status = in.readUTF();
```

```
    if(status.equals("ERROR")){  
        verifyLabel.setText("Exam Not Found");  
        return;  
    }
```

```
    int len = in.readInt();  
    encryptedQP = new byte[len];  
    in.readFully(encryptedQP);
```

```
    encryptedBox.setText(Base64.getEncoder().encodeToString(encryptedQP));
```

```
    int sigLen = in.readInt();  
    signature = new byte[sigLen];  
    in.readFully(signature);
```

```
    signatureBox.setText(Base64.getEncoder().encodeToString(signature));
```

```
    String serverPub = in.readUTF();
```

```
    serverPublicKeyBox.setText(serverPub);
```

```
    byte[] keyBytes = Base64.getDecoder().decode(serverPub);
```

```
    KeyFactory kf = KeyFactory.getInstance("RSA");
```

```
    serverPublicKey =  
        kf.generatePublic(new X509EncodedKeySpec(keyBytes));
```

```
    verifyLabel.setText("Encrypted Exam Received");
```

```

        socket.close();

    }catch(Exception e){
        verifyLabel.setText("Connection Error");
    }
}

private void decryptAndVerify(){

    try{

        Cipher decryptCipher = Cipher.getInstance("RSA");

        decryptCipher.init(Cipher.DECRYPT_MODE,clientKeyPair.getPrivate());

        byte[] decrypted = decryptCipher.doFinal(encryptedQP);

        String exam = new String(decrypted);

        decryptedBox.setText(exam);

        Cipher verifyCipher = Cipher.getInstance("RSA");

        verifyCipher.init(Cipher.DECRYPT_MODE,serverPublicKey);

        byte[] verified = verifyCipher.doFinal(signature);

        String original = new String(verified);

        if(original.equals(exam)){

            verifyLabel.setText("Signature Verified - Exam Authentic");

        }else{

            verifyLabel.setText("Exam Paper Tampered");

        }

    }catch(Exception e){

        verifyLabel.setText("Verification Failed");

    }
}

public static void main(String[] args){

    SwingUtilities.invokeLater(RSAExamClient::new);
}

```


}

Output:

RSA Algrithm Client

Exam Request

Class

10

Course Code

CS101

Semester

1

Request Exam

Decrypt & Verify

Client Public Key

MIIBjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEApo9RRnBkFzwpMSjINzGqMDK8Uz24KUMNm4681fDWw02A46enSgfjkhRetp

AuUIPvp+OOGGrzluBsEjDGjUajhlpjcxgEpWojlUTENwzecP9+L+xVdLiop933qGrWkzeOvDFhbympe2jKjVUO+FC+FwGkc52MaGgIVgw==

S7HhVbb56VcyztNm8DY2O6vPRv1ImwzbXcd5mhZBBCLMuOz6urjBXgp5aVAUt+9GcYTUWWLk11PWpHDICqBj0rz5WqgQPLDEmZr0Q==

wKAFdW+n8rNBHwVB7a6PJTCHqW3cCNKUpoYNHf2qbxuY/pu/i6E30D9/V8ZOSV3E6YgkUv45+IoMvdd0//CGTlkgQubjGfwiDAQAB

Decrypted Question Paper

Q1. What is java?
Q2. Explain OOP.
Q3. What is JVM?

Signature Verified - Exam Authentic

RSA Algorithm

Server Public Key

MIIBjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEA04BaAp7grfFBh4arJubhs1V4Aby/GK/KFM0cZy/DhAY3eBgsUK7P/wEccC

Server Activity Log

Server RSA Keys Generated
Server Started on Port 8080
Client Connected
Received Client Public Key
Generating Digital Signature using Server Private Key
Encrypting Exam Paper using Client Public Key
Encrypted Question Paper Sent to Client

Start Server (Port 8080)